

DATA-STRUCTURES & ALGORITHMS

with

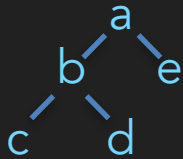
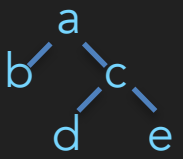
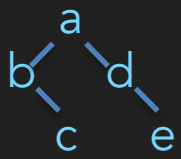
Manoj Prabhakaran

Lecture 14

More on BST

Binary Search Tree (BST)

- Recall: For a general binary tree, pre-order sequence is not sufficient for reconstructing the tree (unless leaves are printed)

- E.g., pre-order sequence `a b c d e` for  and  and 

- But a BST can be uniquely reconstructed from its pre-order sequence
 - Pre-order sequence uniquely determines a BST
 - There is $O(n)$ time algorithm to generate this BST from the sequence

BST Fixed by Pre-order Sequence

- Why does the pre-order sequence uniquely determine a BST?
- Let's try an example:

7.2 3.4 1.2 0.5 2.3 5.1 4.8 6.0 11.3 9.9 8.1 10.0 13.7 12.0 14.6

- Pre-order sequence has the form:

root pre-ord. seq. of left subtree pre-ord. seq. of right subtree

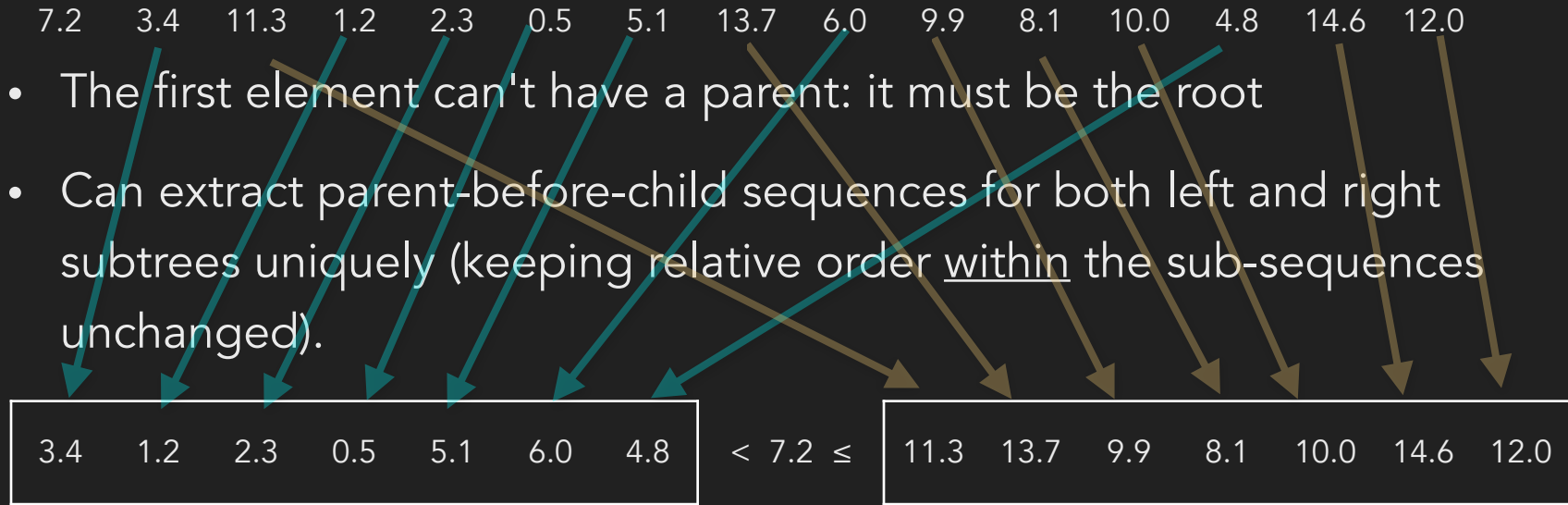
- So 7.2 is the root. Which ones are to its left?
 - They are exactly the ones < 7.2

7.2	3.4	1.2	0.5	2.3	5.1	4.8	6.0	11.3	9.9	8.1	10.0	13.7	12.0	14.6
-----	-----	-----	-----	-----	-----	-----	-----	------	-----	-----	------	------	------	------

- Inductively each of these two pre-order sub-sequences have a unique BST
- So only one BST possible for a given pre-order sequence

BST Fixed by Pre-order Sequence

- Only one BST possible for a given pre-order sequence.
- In fact, any parent-before-child sequence uniquely determines the BST



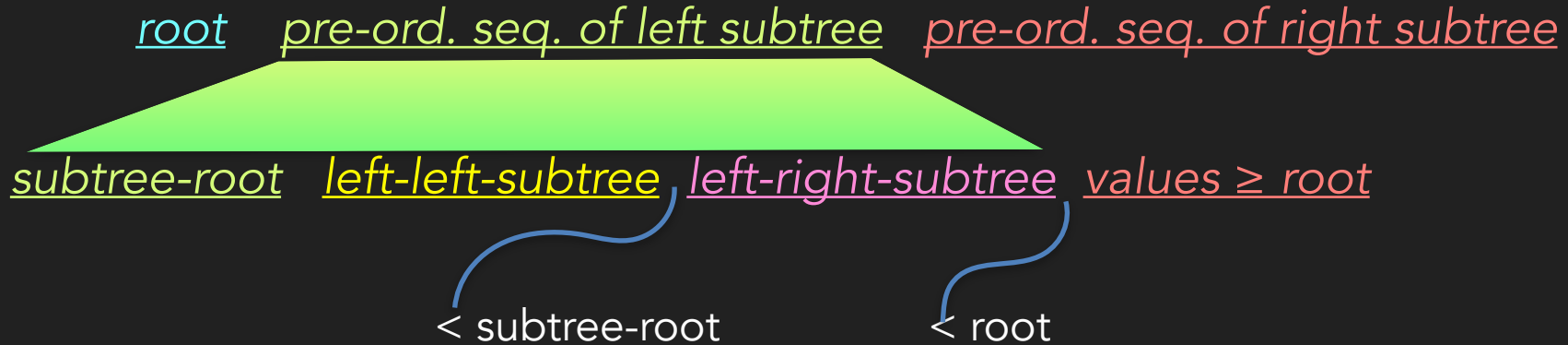
- Inductively, unique BST for these two subtrees

BST From Pre-order Sequence

- How easily can we reconstruct the tree from a pre-order sequence?
- Just insert the elements into the tree in that order!
 - We only rely on the fact that it is a parent-before-child sequence
 - For a BST constructed by inserting elements $x_1, x_2, \dots, x_n$ in order (using the insertion algorithm from Lecture 12), that sequence is a parent-before-child sequence
 - Why? Once inserted, a node's parent doesn't change. The parent was already in the tree means it preceded this child.
 - And this is the unique BST with that sequence as a parent-before-child sequence
- Time is $O(nh)$, where h is the height of the tree

BST From Pre-order Sequence

- Can we reconstruct the BST in $O(n)$ time?
 - Not from general parent-before-child sequences!
(Turns out the best possible is $\Theta(n \log n)$ time)
 - But we can do it for pre-order sequences in $O(n)$ time!
- Idea: Recall we can determine the boundary of a sub-tree's sub-sequence



BST From Pre-order Sequence

```
#include <queue>
#include <limits> // for MAX
const int MAX = std::numeric_limits<int>::max(); // beyond values allowed
struct node{int key; node *parent, *left=nullptr, *right=nullptr};
// Takes reference to a queue which is popped by recursive calls
node* makeBST(std::queue<int>& preseq, node* parent=nullptr, int bound=MAX) {
    if(preseq.empty() || preseq.front() >= bound) return nullptr;
    int key = preseq.front(); preseq.pop();
    node* root = new node{key, parent};
    root->left = makeBST(preseq, root->key), // root's key is bound for left child
    root->right = makeBST(preseq, root, bound); // send given bound to right child
    return root;
}
```

Left subtree keys already popped
by the previous call, since the
queue is passed by reference

Attendance Break!

Login using CSE LDAP id at

https://www.cse.iitb.ac.in/course_vm/cs213

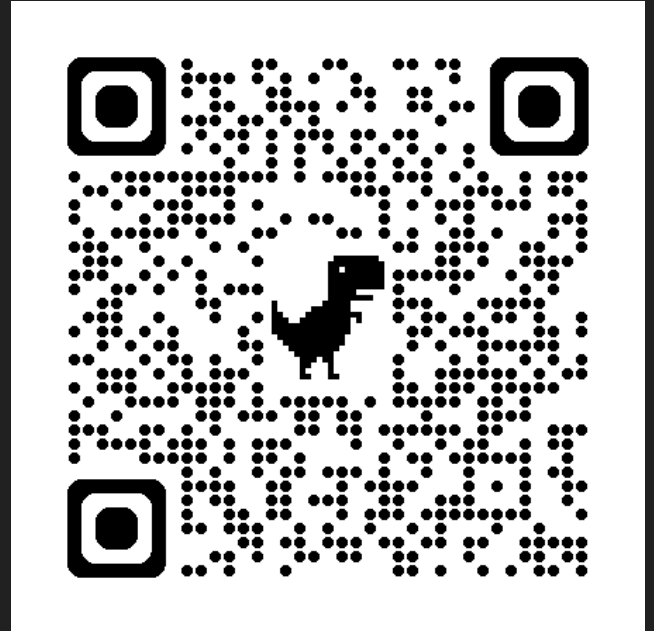
Read the question, choose the answer and submit.

The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

**If you are not in the classroom, a report will
be sent to DDAC!**



BST in C++ STL

- Many containers in C++ STL internally use a BST
- `std::set`
 - Keys can be inserted, searched for, retrieved. Also can iterate through the elements in the set.
 - Allows only one copy of a key (inserting again has no effect)
- `std::map`
 - Holds "key-value pairs." Search by key, retrieve key and value.
 - Implemented just like `std::set`, but nodes have "payloads"
 - Requires unique keys (can replace a value by inserting again)
- `std::multiset` and `std::multimap`
 - Like `std::set` and `std::map`, but allows multiple entries with the same key

BST in C++ STL

- These containers can hold any data type which has `operator<`
 - Other comparison operations can be defined in terms of `<`
 - Instead of `a >= b`, use `!(b<a)`
 - Instead of `a == b`, use `!(a<b || b<a)`
- The containers expose in-order traversal
 - `next_inorder()` is implemented as `operator++` for an iterator for the class
 - Also supports decrementing, `operator--`
- The internal state of the tree is not revealed by the interface
 - In particular, no pre-order or post-order traversal
- These containers offer $O(\log n)$ -time search, insert and delete
 - Keeps the height of the tree $O(\log n)$ by "*balancing*" [later]